

Creating SVG Wave Dividers for HTML

A Complete Guide

Introduction

SVG wave dividers are a modern web design element used to create visually appealing transitions between sections of a webpage. This guide will teach you how to create and customize your own SVG wave dividers for HTML projects.

Basic Implementation

Here's how to add a basic wave divider to your HTML:

```
html
```

```
<div style="position: relative; width: 100%; height: 120px; overflow: hidden;">
  <svg viewBox="0 0 1200 120" preserveAspectRatio="none" style="position: absolute; width: 100%
  <path d="M0,0 C150,60 350,0 500,60 C650,120 800,60 1000,100 C1050,120 1150,60 1200,80 L1200
</svg>
</div>
```

Understanding the SVG Code

1. SVG Container:

- `viewBox="0 0 1200 120"` - Defines the SVG's coordinate system (width 1200, height 120)
- `preserveAspectRatio="none"` - Allows the SVG to stretch fully across its container
- `style="width: 100%; height: 100%;"` - Makes the SVG fill its parent container

2. Path Element:

- The `d` attribute contains the wave's path instructions
- The `fill` attribute sets the color of the wave

How to Customize Wave Dividers

1. Changing the Wave Shape

The path is defined with Cubic Bezier curves (`C` commands). The format is:

```
M0,0 C x1,y1 x2,y2 x3,y3 C x4,y4 x5,y5 x6,y6...
```

- `M0,0` - Moves to starting point (0,0)
- `C 150,60 350,0 500,60` - Creates the first curve
 - Control point 1: (150,60)
 - Control point 2: (350,0)
 - End point: (500,60)

Adjust these values to change the wave pattern:

- Increasing y-values makes waves taller
- Decreasing y-values makes waves flatter
- Adding more C commands creates more waves

2. Changing Colors

Simply modify the `fill` attribute value:

html

```
<path d="..." fill="#4B89DC"/>
```

You can use any color format: named colors, hex codes, rgba, etc.

Wave Variation Examples

1. Simple Wave:

html

```
<path d="M0,0 C300,40 600,0 900,40 C1050,60 1150,20 1200,30 L1200,60 L0,60 Z" fill="#3498DB"/>
```



2. Multi-Wave (more detailed):

html

```
<path d="M0,0 C150,20 250,0 300,10 C400,30 450,0 500,10 C600,30 650,0 700,10 C800,30 850,0 900,10 C1000,30 1050,0 1100,10 L1100,60 L0,60 Z" fill="#3498DB"/>
```



3. Layered Wave (inverted):

html

```
<path d="M0,30 C150,10 350,50 500,30 C650,10 800,50 1000,20 C1050,15 1150,40 1200,30 L1200,60 L
```



4. **Curved Wave** (using quadratic curves):

html

```
<path d="M0,60 Q300,0 600,60 Q900,120 1200,60 L1200,120 L0,120 Z" fill="#2ECC71"/>
```

Creating a Wave Generator

Here's a complete HTML page with controls to adjust a wave divider in real-time:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SVG Wave Generator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 900px;
      margin: 0 auto;
      padding: 20px;
    }
    .controls {
      display: grid;
      grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
      gap: 15px;
      margin-bottom: 20px;
    }
    .control-group {
      margin-bottom: 10px;
    }
    label {
      display: block;
      margin-bottom: 5px;
    }
    .preview {
      position: relative;
      width: 100%;
      height: 150px;
      overflow: hidden;
      border: 1px solid #ccc;
      margin-bottom: 20px;
    }
    .code-output {
      background: #f4f4f4;
      padding: 15px;
      border-radius: 4px;
      overflow: auto;
    }
  </style>
</head>
<body>
```

```

<h1>SVG Wave Divider Generator</h1>

<div class="controls">
  <div class="control-group">
    <label for="height">Height (px)</label>
    <input type="range" id="height" min="50" max="300" value="150" oninput="updateWave()">
    <span id="height-value">150</span>
  </div>

  <div class="control-group">
    <label for="amplitude">Wave Amplitude</label>
    <input type="range" id="amplitude" min="0" max="100" value="30" oninput="updateWave()">
    <span id="amplitude-value">30</span>
  </div>

  <div class="control-group">
    <label for="waves">Number of Waves</label>
    <input type="range" id="waves" min="1" max="8" value="3" oninput="updateWave()">
    <span id="waves-value">3</span>
  </div>

  <div class="control-group">
    <label for="color">Color</label>
    <input type="color" id="color" value="#4B89DC" oninput="updateWave()">
  </div>
</div>

<div class="preview">
  <svg id="wave-svg" viewBox="0 0 1200 150" preserveAspectRatio="none" style="position: absol
  <path id="wave-path" d="" fill="#4B89DC"></path>
  </svg>
</div>

<h3>Generated Code:</h3>
<pre class="code-output" id="code-output"></pre>

<script>
  function updateWave() {
    const height = document.getElementById('height').value;
    const amplitude = document.getElementById('amplitude').value;
    const waves = document.getElementById('waves').value;
    const color = document.getElementById('color').value;

    // Update value displays

```

```

..... document.getElementById('height-value').textContent = height;
..... document.getElementById('amplitude-value').textContent = amplitude;
..... document.getElementById('waves-value').textContent = waves;
.....
..... // Update SVG
..... const svg = document.getElementById('wave-svg');
..... svg.setAttribute('viewBox', `0 0 1200 ${height}`);
.....
..... // Generate path
..... let path = `M0,${height * 0.8}`;
.....
..... const width = 1200;
..... const segmentWidth = width / waves;
.....
..... for (let i = 0; i < waves; i++) {
.....     const x1 = i * segmentWidth;
.....     const x2 = (i + 0.5) * segmentWidth;
.....     const x3 = (i + 1) * segmentWidth;
.....
.....     // Calculate y positions with random variation
.....     const yStart = height * 0.8;
.....     const yMid = yStart - amplitude;
.....
.....     path += ` C${x1 + segmentWidth/4},${yStart} ${x2 - segmentWidth/4},${yMid} ${x2},${yMid}
.....     path += ` C${x2 + segmentWidth/4},${yMid} ${x3 - segmentWidth/4},${yStart} ${x3},${yStart}
..... }
.....
..... path += ` L${width},${height} L0,${height} Z`;
.....
..... // Update path
..... const wavePath = document.getElementById('wave-path');
..... wavePath.setAttribute('d', path);
..... wavePath.setAttribute('fill', color);
.....
..... // Generate code
..... const htmlCode = `
<div style="position: relative; width: 100%; height: ${height}px; overflow: hidden;">
<svg viewBox="0 0 1200 ${height}" preserveAspectRatio="none" style="position: absolute; width: 100%; height: 100%;>
<path d="${path}" fill="${color}" />
</svg>
</div>`;
.....
..... document.getElementById('code-output').textContent = htmlCode;
..... }

```

```

</script>
    // Initialize
    updateWave();
</script>
</body>
</html>

```

Advanced Techniques

1. Creating Inverted Waves

Start your path at the bottom instead of the top and curve upward.

2. Multiple Layers

Use multiple SVG paths with different colors and opacity to create a layered effect:

```

html

<div style="position: relative; width: 100%; height: 150px; overflow: hidden;">
  <svg viewBox="0 0 1200 150" preserveAspectRatio="none" style="position: absolute; width: 100%
  <!-- Bottom wave -->
  <path d="M0,30 C300,80 600,10 1200,70 L1200,150 L0,150 Z" fill="#4B89DC" opacity="0.7"/>
  <!-- Top wave -->
  <path d="M0,60 C200,20 500,90 1200,20 L1200,150 L0,150 Z" fill="#4B89DC" opacity="0.3"/>
</svg>
</div>

```

Creative Animation Ideas

1. Horizontal Wave Motion

Create an effect of waves moving from side to side:

html

```
<style>
  @keyframes wave {
    0% { transform: translateX(0); }
    100% { transform: translateX(-50%); }
  }

  .wave-container {
    position: relative;
    width: 100%;
    height: 150px;
    overflow: hidden;
  }

  .wave {
    width: 200%; /* Double width to allow animation */
    animation: wave 15s linear infinite;
  }
</style>

<div class="wave-container">
  <svg class="wave" viewBox="0 0 2400 150" preserveAspectRatio="none" style="position: absolute;
  <path d="M0,30 C300,80 600,10 1200,70 C1800,10 2100,80 2400,30 L2400,150 L0,150 Z" fill="#4
  </svg>
</div>
```

2. Rising/Falling Wave Animation

Make waves that rise up or sink down:

html

```
<style>
  @keyframes rise {
    0% { transform: translateY(0); }
    50% { transform: translateY(-15px); }
    100% { transform: translateY(0); }
  }

  .wave-container {
    position: relative;
    width: 100%;
    height: 150px;
    overflow: hidden;
  }

  .wave {
    position: absolute;
    width: 100%;
    height: 100%;
    animation: rise 8s ease-in-out infinite;
  }
</style>

<div class="wave-container">
  <svg class="wave" viewBox="0 0 1200 150" preserveAspectRatio="none">
    <path d="M0,60 C300,0 600,100 900,30 C1050,0 1150,30 1200,50 L1200,150 L0,150 Z" fill="#4B8E3B">
  </svg>
</div>
```



3. Reveal Animation on Hover

This animation reveals an image when hovering by moving the wave divider downward:

html

```
<style>
... .image-container {
...   position: relative;
...   width: 100%;
...   height: 400px;
...   overflow: hidden;
... }
...
... .image-container img {
...   width: 100%;
...   height: 100%;
...   object-fit: cover;
... }
...
... .wave-overlay {
...   position: absolute;
...   top: 0;
...   left: 0;
...   width: 100%;
...   height: 100%;
...   background-color: rgba(255, 255, 255, 0.9);
...   transition: transform 0.6s ease-out;
... }
...
... .wave-divider {
...   position: absolute;
...   bottom: 0;
...   left: 0;
...   width: 100%;
...   height: 100px;
...   overflow: hidden;
...   transform: translateY(-50%);
...   transition: transform 0.6s ease-out;
... }
...
... .image-container:hover .wave-overlay {
...   transform: translateY(100%);
... }
...
... .image-container:hover .wave-divider {
...   transform: translateY(0);
... }
</style>
```

```

<div class="image-container">
  
  <div class="wave-overlay">
    <div class="wave-divider">
      <svg viewBox="0 0 1200 120" preserveAspectRatio="none" style="height: 100%; width: 100%;"
      <path d="M0,0 C150,60 350,0 500,60 C650,120 800,60 1000,100 C1050,120 1150,60 1200,80 L
      </svg>
    </div>
  </div>
</div>

```

4. Image Reveal with Wave Animation (Your Requested Effect)

This implementation creates a wave divider that covers half of an image and moves down to reveal the full image on hover:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Wave Image Reveal</title>
  <style>
    .wave-image-container {
      position: relative;
      width: 100%;
      max-width: 600px;
      height: 400px;
      overflow: hidden;
      margin: 0 auto;
    }

    .wave-image {
      width: 100%;
      height: 100%;
      object-fit: cover;
    }

    .wave-cover {
      position: absolute;
      top: 0;
      left: 0;
      width: 100%;
      height: 50%; /* Covers half the image */
      background-color: #fff;
      transition: height 0.5s ease-in-out;
    }

    .wave-bottom {
      position: absolute;
      bottom: 0;
      left: 0;
      width: 100%;
      height: 80px;
      overflow: hidden;
    }

    .wave-svg {
      position: absolute;
```

```

..... bottom: 0;
..... left: 0;
..... width: 100%;
..... height: 100%;
..... }
.....
..... /* The hover effect */
..... .wave-image-container:hover .wave-cover {
..... height: 0; /* Slides the cover up to reveal the image */
..... }
.. </style>
</head>
<body>
.. <div class="wave-image-container">
..... 
..... <div class="wave-cover">
..... <div class="wave-bottom">
..... <svg class="wave-svg" viewBox="0 0 1200 120" preserveAspectRatio="none">
..... <path d="M0,0 C150,40 350,0 500,40 C650,80 800,40 1000,60 C1050,70 1150,40 1200,50 L1
..... </svg>
..... </div>
..... </div>
.. </div>
</body>
</html>

```

5. Interactive Wave with Cursor

Create a wave that responds to mouse movement:

html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Wave</title>
  <style>
    .interactive-wave-container {
      position: relative;
      width: 100%;
      height: 200px;
      overflow: hidden;
      background-color: #f5f5f5;
    }
  </style>
</head>
<body>
  <div class="interactive-wave-container" id="wave-container">
    <svg viewBox="0 0 1200 200" preserveAspectRatio="none" style="width: 100%; height: 100%;">
      <path id="wave-path" d="" fill="#4B89DC"></path>
    </svg>
  </div>

  <script>
    const container = document.getElementById('wave-container');
    const wavePath = document.getElementById('wave-path');

    const width = 1200;
    const height = 200;

    // Initial wave
    updateWave(width / 2, height / 2);

    // Update wave on mouse move
    container.addEventListener('mousemove', (e) => {
      const rect = container.getBoundingClientRect();
      const x = ((e.clientX - rect.left) / rect.width) * width;
      const y = ((e.clientY - rect.top) / rect.height) * height;
      updateWave(x, y);
    });

    function updateWave(mouseX, mouseY) {
      // Calculate wave amplitude based on mouse position
      const amplitude = (mouseY / height) * 80;

```

```

.....
        // Generate path
..... let path = `M0,${height}`;
.....
        for (let i = 0; i <= width; i += 50) {
..... // Calculate distance from mouse
..... const distX = Math.abs(mouseX - i);
..... const influence = Math.max(0, 1 - (distX / 300));
.....
..... // Calculate y position with influence from mouse
..... const y = height - (amplitude * influence * Math.sin(i / 100));
.....
..... if (i === 0) {
..... path += ` L${i},${y}`;
..... } else {
..... path += ` Q${i - 25},${y - 10} ${i},${y}`;
..... }
..... }
.....
..... path += ` L${width},${height} L0,${height} Z`;
..... wavePath.setAttribute('d', path);
..... }
.. </script>
</body>
</html>

```

Conclusion

SVG wave dividers add visual interest to your web designs and are highly customizable. With the techniques in this guide, you can create unique wave patterns that match your design needs. Experiment with different path values, colors, and animations to create your perfect wave divider.